

Training a Physics Informed Neural Network on projectile motion with drag

Tom Bensky

Department of Physics

California State University San Luis Obispo

San Luis Obispo, CA 93407, USA

tbensky@calpoly.edu

Abstract—Here we present a neural network trained on the dynamics of the classic physics problem of projectile motion with drag. In the spirit of a Physics Informed Neural Network (or PiNN), the training set consists only minimal data about the position and velocity of the projectile with the majority of its dynamics trained against the differential equation for the motion. We show how a ~ 1 million parameter neural network can learn the dynamics of this system over a wide domain, showing excellent interpolation abilities within the training domain, and extrapolation abilities beyond it.

Index Terms—Projectile motion, drag, neural network, physics informed neural network, differential equation, numerical modelling

I. INTRODUCTION

There is no shortage of articles in the physics (educational) literature discussing projectile motion that includes the quadratic drag force due to air resistance [1], with the required numerical integration the common theme. Over the decades, these articles also seem to be a proxy for the latest technology available, from computing trajectories on the first home computers [2] to comparisons with precision projectile launchers [3] to interactive web animations [4] to video analysis [5]. This article presents still another example of projectile motion with drag, using more current technology: a neural network.

Neural networks are a mathematical construct from the world of artificial intelligence, whose full structure and latest developments are described elsewhere [6]. Briefly, they are a mathematical entity, consisting of many “neurons” organized into layers, with the neurons in each layer interconnected to and feeding each neuron in the subsequent layer (see Fig. 1). A neuron computes a weighted sum of its inputs (see Fig. 2), which is passed through an activation function, then forward as an input to another downstream neuron.

The interconnections are the weights used to compute the sum of a receiving neuron and collectively are the free parameters of the network. The width (number of neurons in a given layer) and depth (number of layers) of a neural network are arbitrary, and can result in a very large number of free parameters. Data flow in Fig. 1 is from left to right, meaning inputs are applied to the four leftmost neurons, with sums and activations computed to flow toward the right, until outputs can be extracted from the five rightmost neurons.

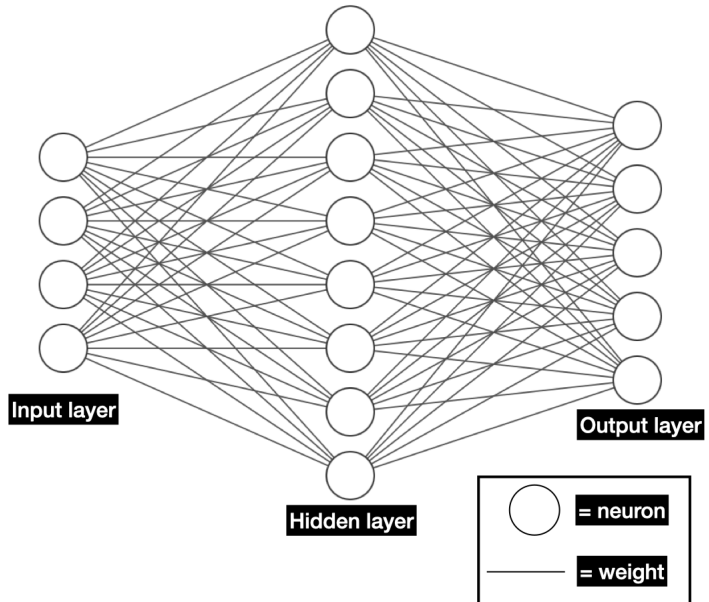


Fig. 1. A neural network, showing the interconnected layers of neurons. It is read from left to right. This one has four input neurons on the left, one hidden layer of eight neurons, and one output layer of five neurons on the right. (Generated from <https://alexlenail.me/NN-SVG/>).

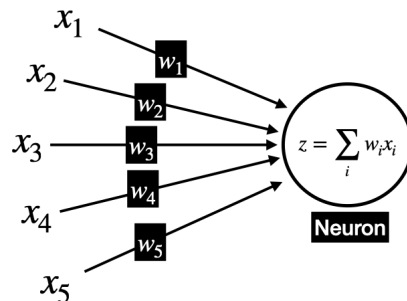


Fig. 2. A neuron as used in a neural network. Here the x_i are the inputs and the w_i are the weights. The weighted sum z , is the net input to the neuron.

Training a neural network consists of presenting it with a set of corresponding data pairs, with the hope that repeated presentations of the input of each pair, combined with continual weight updates, will eventually cause the network to produce

the output of each pair. Generally, an untrained network will not produce needed outputs, so an error is computed based on the difference between the output of the network and the required output. The error is sent backward (toward the left) through the network and is used to adjust the weights using an algorithm called backpropagation. [7] This process is repeated until the error (or loss) [8] across the entire data set is minimal.

II. A PHYSICS INFORMED NEURAL NETWORK

A Physics Informed Neural Network (or PiNN) [9] is a neural network, like the one described above. The difference is how it is trained. Typically, training a neural network acts to minimize the loss solely due to data pairs. A PiNN however is trained only on a limited number of data pairs known about a system [10], with the differential equation of the system used to supplement the training in areas of the domain where data is lacking. Below a PiNN trained for projectile motion with quadratic drag is presented.

III. A PiNN FOR PROJECTILE MOTION WITH DRAG

As this article is meant to be a demonstration of a PiNN to a familiar system, we begin by presenting a traditional numerical integration of a projectile with drag. These components for acceleration [3]

$$a_x = -\frac{C}{m}vv_x \quad (1)$$

and

$$a_y = -g - \frac{C}{m}vv_y, \quad (2)$$

were used, where C is the drag coefficient, m is the mass, g is the acceleration due to gravity, v is the projectile's speed (and v_x or v_y the components thereof). Using these, Fig. 3 was produced with a numerical integration [11] using an initial speed of 30 m/s at an angle of 70° with the horizontal with $g = 9.8 \text{ m/s}^2$, $C = 0.01$ and $m = 1 \text{ kg}$. The three larger plotting symbols represent the limited of data that will be presented to the PiNN during training (see below).

IV. TRAINING THE PiNN

To train the PiNN, a neural network was created using the PyTorch [12] framework. It consists of a single input neuron representing time, with four output neurons representing x , y , v_x and v_y of the projectile at that time. The network used is shown in Fig. 4, which consists of five hidden (or "deep") layers, each consisting of 512 neurons. Thus this PiNN has 1,053,700 free parameters [13].

The loss function of the network consists of two parts, one for the data and one for the differential equation. The loss due to the data is given by

$$L_{data} = \sum_{i=1}^4 (D_i - O_i)^2, \quad (3)$$

where the sum is over all four output neurons of the network. Here D_i is the required data (large dots in Fig. 3) value for the projectile at neuron i when time t_i is applied as input to

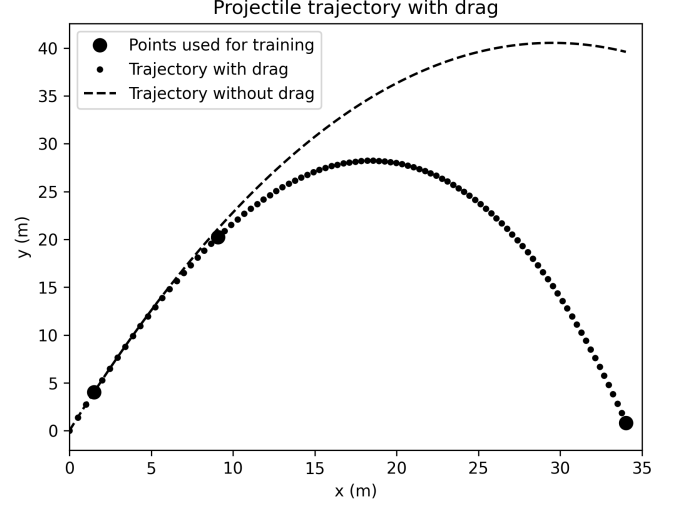


Fig. 3. Numerical integration of projectile motion with quadratic drag (dots) vs. drag free trajectory (dashed). The three highlighted points are the only data allowed during training.

the network. The observed output from the network after a forward pass, as seen on output neuron i is given by O_i .

The loss due to the differential equation is given by

$$L_{DE} = (a_x + \frac{C}{m}vv_x) + (a_y + g + \frac{C}{m}vv_y), \quad (4)$$

with the total training loss of the network given by

$$L_{total} = L_{data} + L_{DE}. \quad (5)$$

Training proceeds by presenting the network with the time values of each of the three data points highlighted in Fig. 3 and tabulated in Table 1. After a forward pass of these times, L_{data} is computed directly from the outputs of the network.

Next, the additional times of 0.25, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0, 6.0 seconds are presented to the network for which L_{DE} is computed from the resulting network outputs[14].

With L_{data} and L_{DE} , L_{total} may now be computed, which was logged as the network trained as shown in Fig. 4. Here about 7,000 epochs[15] were required before a minimal and converging loss was seen. At this point, the network is considered trained, and is now able to produce outputs 1) matching those required by the input data points, and 2) that were previously unknown, but are consistent with the differential equation.

The trajectory found from the trained network is shown as the solid line in Fig. 5 with further network interrogation shown in Fig. 6. We emphasize that the curves in Figs. 5 and 6 were all found by only interrogating the trained PiNN. To generate these curves, the PiNN was presented with a value of time, after which a forward pass through the network was executed. Values plotted were taken directly from the output neurons (see Fig. 7 for mappings).

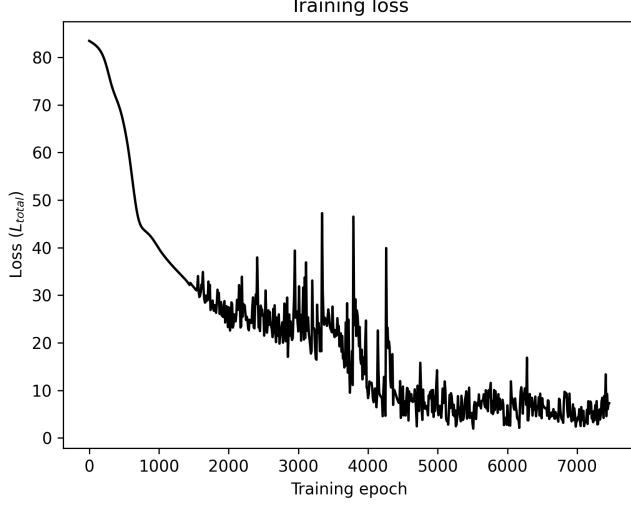


Fig. 4. Training loss for the PiNN as a function of epoch.

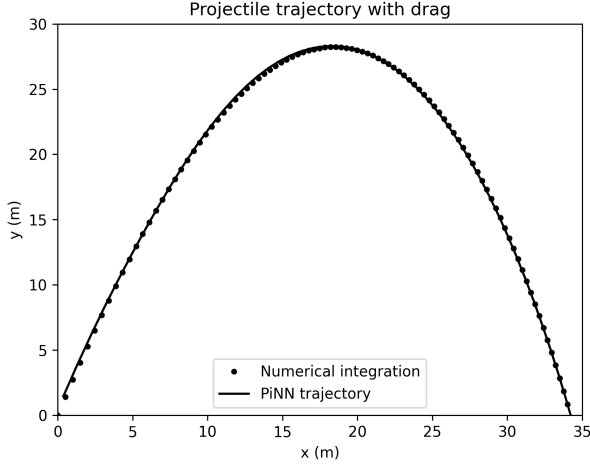


Fig. 5. The solid line is the PiNN predicted trajectory. Dots are the numerical integration results.

V. DISCUSSION

The training cycle of the PiNN used the three data points and the differential equation to find values for the weights in the network that allowed it to produce the solid curve in Fig. 5. The PiNN in this case is functioning as a curve fit with over a million free parameters. It has good interpolation abilities, as the curve in Fig. 5 is made up of approximately 10 times the number of points used for training.

We emphasize that a neural network is strictly a numerical model. This one doesn't somehow "know" about projectile motion. Although why neural networks produce results is generally poorly understood [16], the how is not: the error correction algorithm of backpropagation is based on computing the gradient of the entire network, whose values have closed form expressions, derivable from use of the chain rule from

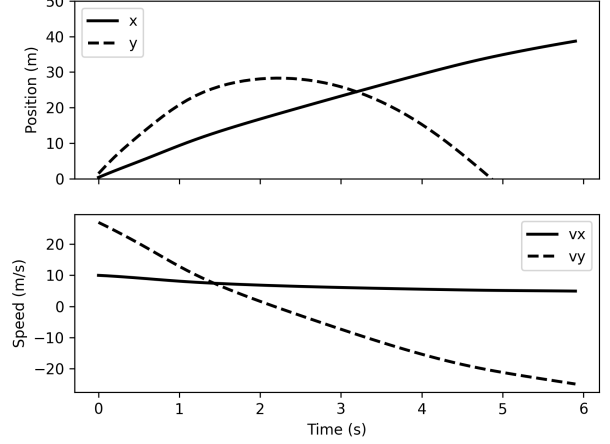


Fig. 6. Interrogating the PiNN and showing its x and y positions and speeds.

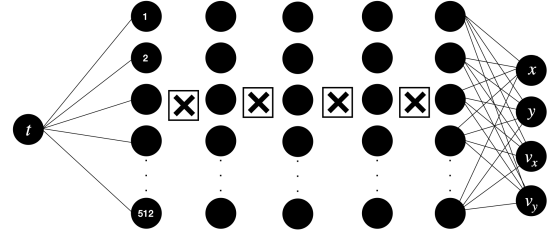


Fig. 7. The neural network used here. One (scalar) input neuron representing time (t) feeds five layers, each containing 512 neurons, followed by a four neuron output layer (vector) yielding the dynamical parameters of the projectile. The $\boxtimes$ represents fully connected layers.

calculus [6]. Likewise, the mathematics of a forward pass are straightforward.

The PiNN presented here was given enough information to drive the training process to find a combination of weights that minimized the network loss for the entire projectile motion training data set discussed. This was sufficient to reproduce the numerical integration results. Thus, training a neural network is a numerical optimization that (like all optimizations) must have sufficient guidance in order to work. For example, if L_{DE} is omitted from the loss, the network will find a solid curve that passes through the three data points, missing the full trajectory (see Fig. 8).

VI. CONCLUSIONS

We have shown how a neural network can “upscale” a coarse set of knowns into a fine scale result, producing details of a system not available in data alone.

Finally, we find it interesting to interrogate the network while it trains. A video showing the network's best guess at a trajectory as a function of training epoch is available [17], and shows the network struggling to learn the projectile motion, for which it eventually succeeds.

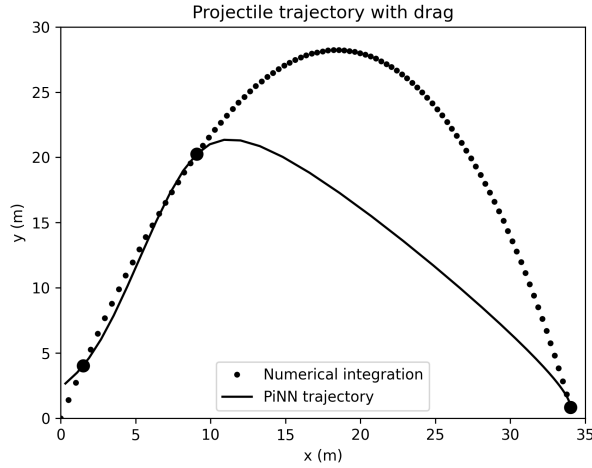


Fig. 8. Trajectory found by the neural network (solid curve) with L_{DE} turned off (i.e. the network is trained only on the three data points).

VII. ACKNOWLEDGEMENTS

The author wishes to thank the Bailey College of Science and Mathematics for granting the author a 2023-2024 academic year sabbatical when much of this work was done, during a most stimulating visit with the Evolutionary Computation and Machine Learning group at Victoria University in Wellington, New Zealand. Thank you also to M. Moelter of the Cal Poly Physics Department for helpful suggestions and for proofreading this article.

REFERENCES

- [1] One needs only to search for “projectile motion with drag” at any journal repository.
- [2] Peter Brancazio, “Trajectory of a fly ball,” *Phys. Teach.* 23 20-23 (January 1985).
- [3] Pirooz Mohazzabi, “When Does Air Resistance Become Significant in Projectile Motion?” *Phys. Teach.* 56 168-169 (March 2018).
- [4] See the projectile PhET at https://phet.colorado.edu/sims/html/projectile-motion/latest/projectile-motion_all.html.
- [5] Pradita Wijaya, Umar Fauzi, and Fasya Latief, “A simple determination of air drag using video tracker and modifiable projectile launcher,” *Phys. Educ.* 54 055009 (2019).
- [6] Stuart Russell and Peter Norvig, 2022. *Artificial Intelligence: A Modern Approach*, Pearson, Chapter 22.
- [7] See Ref. [6], Sect. 22.1.2, p. 805.
- [8] Taylor, J.R. (1997) *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. 2nd Edition, University Science Books, Sausalito, p. 8.2, especially Eqn. 8.5.
- [9] Luis Medrano Navarro et al, “Solving differential equations with deep learning: a beginner’s guide” 2024 *Eur. J. Phys.* 45 015803.
- [10] Limited data can be a due to an experimental or theoretical system where data is simply difficult to obtain. Reasons may include computational or experimental limitations, expense, time required, etc.
- [11] We tried both simple Euler steps (with a small step size) and `ode45` in Matlab (<https://mathworks.com>) with no noticeable difference in the results.
- [12] See PyTorch at <https://www.pytorch.org>. The code for this work can be found here https://github.com/tbensky/PiNN_Projectile.
- [13] There are no rules for the size of a neural network. We were guided by S. Wang, S. Sankaran, H. Wang, and P. Perdikaris, “An expert’s guide to training physics-informed neural networks,” arXiv preprint arXiv:2308.08468, 2023.

- [14] Values of a_x and a_y needed to find L_{DE} come from auto-differentiation, whereby derivatives of the network outputs with respect to its inputs are computed piecemeal during a forward pass. This is a feature built into PyTorch[12].
- [15] An “epoch” is defined as one cycle where all data are presented to the network, with errors computed, propagated backward and weights adjusted to lower the total network loss.
- [16] Understanding why neural networks work falls under the AI-topic of “interpretability.”
- [17] See: <https://youtu.be/5bleRneG74E?si=Zax7fC4DPK4gKX5f>.